

System Test Report (STR)

Projekt 6: API für den Digitalen Produktpass (DPP) im BaSyx Framework

Customer

Name	Mail
Markus Rentschler	rentschler@lehre.dhbw-stuttgart.de
Pawel Wojcik	pawel.wojcik@lehre.dhbw-stuttgart.de

Dokumenthistorie

Version	Autor	Datum	Kommentar
1.0	Manuel Lutz	13.03.2026	Erstellung basierend auf der SRS
1.1	Manuel Lutz	13.03.2026	Überarbeitung, Umbenennung und Ergänzung der vorbereiteten Integrationstests
1.2	Manuel Lutz	15.03.2026	Status der vorbereiteten Integrationstests nach Ausführung aktualisiert (12/12 bestanden)
1.3	Manuel Lutz	27.03.2026	Anpassung der Integrationstests auf DIN EN 18222 DPP-Data-Object-Structure Mapping
1.4	Manuel Lutz	30.03.2026	Methoden-/Endpoint-Abgleich mit API-Mapping (PATCH/POST, Date-Query, Namenskonventionen) präzisiert
1.5	Manuel Lutz	09.05.2026	Real-Backend-Test-Struktur implementiert; Mock-Harness und Setup-Skript hinzugefügt; 26/26 Tests mit Umschaltungsmechanismus
1.6	Manuel Lutz	12.05.2026	Struktur, Traceability und Abgrenzung zwischen Systemtest und vorbereiteten Integrationstests präzisiert

Begrifflichkeiten und Abkürzungen

Abkürzung	Bedeutung
DPP	Digitaler Produktpass
AAS	Asset Administration Shell
BaSyx	Open-Source Framework
Submodel	Teilstruktur eines AAS
OpenAPI	Industriestandard zur Beschreibung von APIs
CRS	Customer Requirement Specification
SRS	Software Requirement Specification
STR	System Test Report

Inhaltsverzeichnis

1. Einführung
 2. Testumgebung
 3. Systemtestfälle
 1. Backend-Testfälle
 2. Frontend-Testfälle
 3. Nicht-funktionale Testfälle
 4. Ausführung und Ergebnisse
 1. Ziel und Abgrenzung
 2. API-Integration
 3. Frontend-Backend-Integration
 4. BaSyx-Integration
 5. Artefakte und Ablage
 6. Zusammenfassung
-

1. Einführung

Dieses STR dokumentiert die Systemtests für die Implementierung der DPP-API gemäß DIN EN 18222 im BaSyx Framework. Grundlage sind die funktionalen und nicht-funktionalen Anforderungen aus dem SRS sowie der Testplan aus dem STP.

Da ein Teil der Backend- und BaSyx-Integration noch nicht in der Endausprägung vorliegt, dokumentiert dieses STR sowohl bereits ausgeführte vorbereitete Integrationstests als auch die noch offenen Systemtestanteile.

Der Schwerpunkt liegt auf:

- der Ableitung der Testfaelle aus SRS und STP,
- der Trennung zwischen Mock-basierter Vorbereitung und echter Systemausfuehrung,
- der nachvollziehbaren Dokumentation von Ergebnissen, Status und Grenzen.

2. Testumgebung

- **Zielplattform:** Web-Anwendung im BaSyx-Kontext
- **Frontend:** Vue 3 / Vitest / Vue Test Utils
- **Testausführung:** Vitest im bestehenden Frontend-Projekt
- **Mocking:** `vi.fn()` und `vi.stubGlobal()` für HTTP- und Service-Mocks
- **Referenzdaten:** Beispiel-DPPs auf Basis der in der SRS genannten IDTA-Submodelle
- **Echte Integration:** BaSyx AAS Repository, Submodel Repository, Registry, Discovery
- **Testdatenpflege:** `SOURCE/frontend/aas-web-ui/scripts/setupTestData.mjs`

2.1 Bewertungslogik

Systemtestfaelle werden als Pass bewertet, wenn Funktion, Statuscode und Rückgabeformat dem STP entsprechen. Teilweise ausgeführte oder vorbereitete Faelle werden als Not Executed markiert und gesondert begründet.

3. Systemtestfälle

3.1 Backend-Testfälle

Test-ID	Beschreibung	Schritte	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-BE-01	Erstellung eines DPP	Gültiger DPP-Payload	POST /dpps absenden	Neues DPP-Submodel bzw. DPP wird angelegt	Pass

Test-ID	Beschreibung	Schritte	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-BE-02	Abrufen eines DPP per ID	Vorhandene dppId	GET /dpps/{dppId} absenden	Volle DPP-Daten zurück	Pass
TC-BE-03	Updaten eines DPP per ID	Bestehende dppId , partielle Änderungen	PATCH /dpps/{dppId} absenden	Aktualisierte Daten gespeichert	Not Executed
TC-BE-04	Loeschen eines DPP per ID	Bestehende dppId	DELETE /dpps/{dppId} absenden	DPP nicht mehr auffindbar	Not Executed
TC-BE-05	Abrufen eines DPP per productId	Gültige productId	GET /dppsByProductId/{productId} absenden	Zugeordnetes DPP zurück	Pass
TC-BE-06	Abrufen eines DPP per Datum	Gültige productId und ISO-8601-Datum	GET /dppsByProductIdAndDate/{productId}?date=... absenden	Historische oder aktuelle Version wird geliefert	Pass
TC-BE-07	Abrufen mehrerer DPPs per Liste	Liste von productIds	POST /dppsByProductIds absenden	Liste von DPP-IDs wird geliefert	Pass
TC-BE-08	Registrierung im AAS Registry	Gültiges DPP mit Registry-Referenz	POST /registerDPP ausführen	DPP im Registry auffindbar	Pass
TC-BE-09	Abrufen spezifischer Submodel-Daten	Gültige dppId und elementId	GET /dpps/{dppId}/collections/{elementId} absenden	Gespeicherte Daten zurück	Pass
TC-BE-10	Abrufen eines Elements per elementPath	Gültige dppId und elementPath	GET /dpps/{dppId}/elements/{elementPath} absenden	Element wird korrekt geliefert	Pass
TC-BE-11	Updaten spezifischer Submodel-Daten	Gültige dppId , elementId und Wert	PATCH /dpps/{dppId}/collections/{elementId} absenden	Element aktualisiert	Pass
TC-BE-12	Ungültige Parameter	Falsche dppId oder ungültiger Query-Parameter	Request absenden	Strukturierte 400/404-Fehlerantwort	Pass

3.2 Frontend-Testfälle

Test-ID	Beschreibung	Schritte	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-FE-01	Laden eines DPP	Gültige AAS-/DPP-Referenz	DPP im Viewer öffnen	Volles DPP angezeigt	Pass
TC-FE-02	Navigation innerhalb des DPP	DPP mit mehreren Submodellen	Seitenleiste prüfen und Element wählen	Seitenleiste mit Submodellen sichtbar	Pass
TC-FE-03	Klickbare Navigation	Submodell in Seitenleiste wählen	Informationen anzeigen	Submodell-Informationen werden angezeigt	Pass
TC-FE-04	Highlighting des Submodells	Aktives Submodell wechseln	Hervorhebung prüfen	Aktuelles Submodell ist markiert	Pass
TC-FE-05	Menü oberhalb des Viewers	Viewer mit Navigationsmenü	Modi wechseln	Menü mit DPP-, AAS- und Submodell-Ansicht vorhanden	Pass
TC-FE-06	Informationen zum Produkt	DPP mit Produktmetadaten	Header prüfen	productId, Version und Name werden angezeigt	Pass
TC-FE-07	Anzeige der Kategorien	Submodell mit Kategorien	Darstellung prüfen	Zweispaltige, übersichtliche Darstellung	Pass
TC-FE-08	Fehlende Daten	DPP mit fehlenden optionalen Feldern	Anzeige prüfen	Fehlende Daten werden markiert oder als N/A dargestellt	Pass
TC-FE-09	Tooltips für Daten	Datenfelder mit Zusatzinfos	Tooltip prüfen	Tooltip mit DIN-/IDTA-Informationen sichtbar	Pass
TC-FE-10	Responsives Design	Desktop- und Mobilansicht	Layout prüfen	Responsive Darstellung korrekt	Pass

3.3 Nicht-funktionale Testfälle

Test-ID	Beschreibung	Schritte	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-NFR-01	OpenAPI-Konformität	API gegen Spezifikation validieren	Konform mit Spezifikation	Konformitätsprüfung dokumentiert	Pass
TC-NFR-02	BaSyx-Konformität	API-Calls und Integrationspfade prüfen	Nur BaSyx-Schnittstellen verwendet	BaSyx-Pfade werden verwendet	Pass
TC-NFR-03	Automatisierte Tests	Unit- und Integrationstests ausführen	Tests vorhanden und erfolgreich	Mock-basierte Tests erfolgreich	Pass

4. Ausführung und Ergebnisse

4.1 Ziel und Abgrenzung

Weil das Backend noch nicht in allen Bereichen final vorliegt, wurden systemnahe Integrationstests als vorbereitete Mock-Tests erstellt. Diese Tests prüfen bereits heute:

- Request- und Response-Strukturen,
- die Interaktion des Frontends mit einer DPP-API,
- die Orchestrierung geplanter BaSyx-Integrationen.

Sobald das Backend vollständig bereitsteht, können die Mock-Antworten schrittweise durch echte API-Aufrufe ersetzt werden, ohne die Testfälle zu ändern.

4.2 API-Integration (DIN EN 18222 konform)

Mapping-Referenzstand für die folgenden API-Tests:

- Basis: DPP-Data-Object-Structure **v1 (2026-03-22)**
- Hinweis: **v2 (2026-03-25)** ist im Mapping-Dokument als "in work" gekennzeichnet und daher noch nicht als verbindliche Testbasis verwendet.

Namenskonventionen aus dem Mapping sind bewusst unterschiedlich und werden in den Tests entsprechend abgebildet:

- CreateDPP-Response verwendet **dppID**
- DPP-Identifizier-Listen verwenden **dppId**

Test-ID	Endpoint	Beschreibung	Ziel	Status
IT-API-01	POST /dpps	CreateDPP	Erzeugung eines DPP mit info und submodels (SuccessCreated Response)	Pass
IT-API-02	GET /dpps/{dppld}	ReadDPPById	Abruf vollständiger DPP-Struktur mit payload	Pass
IT-API-03	GET /dppsByProductId/{productId}	ReadDPPByProductId	Abruf eines DPP über productId -Mapping	Pass
IT-API-04	GET /dppsByProductIdAndDate/{productId}?date={ISO-8601}	ReadDPPVersionByProductIdAndDate	Abruf einer spezifischen DPP-Versionierung mit Datum	Pass
IT-API-05	POST /dppsByProductIds	ReadDPPIdsByProductIds	Abruf von DPP-Identifiern über Liste von productIds	Pass
IT-API-06	PATCH /dpps/{dppld}	UpdateDPPById	Update der DPP-Struktur (Partial)	Pass
IT-API-07	DELETE /dpps/{dppld}	DeleteDPPById	Löschen eines DPP (nur statusCode Response)	Pass
IT-API-08	POST /registerDPP	PostNewDPPToRegistry	Registrierung im AAS Registry mit registryIdentifizier	Pass

Test-ID	Endpoint	Beschreibung	Ziel	Status
IT-API-09	GET /dpps/{dppld}/collections/{elementId}	ReadDataElementCollection	Abruf von Submodel-Daten per elementId	Pass
IT-API-10	GET /dpps/{dppld}/elements/{elementPath}	ReadDataElement	Abruf einzelnen Elements per elementPath	Pass
IT-API-11	PATCH /dpps/{dppld}/collections/{elementId}	UpdateDataElementCollection	Update von Submodel-Datensammlungen	Pass
IT-API-12	PATCH /dpps/{dppld}/elements/{elementPath}	UpdateDataElement	Update einzelner Elemente	Pass
IT-API-13	ClientErrorBadRequest	Error Handling	Prüfung fehlerhafter Requests mit ErrorMessage Struktur	Pass
IT-API-14	ClientErrorResourceNotFound	Error Handling	Prüfung auf 404 mit structured error responses	Pass

4.3 Frontend-Backend-Integration (DIN EN 18222 konform)

Test-ID	Beschreibung	Ziel	Status
IT-FB-01	Viewer lädt DPP mit voller Struktur	Prüfung des Ladens von info , submodels und Status-Codes	Pass
IT-FB-02	Navigation lädt Data Element Collection	Prüfung der Datensammlung über /collections/{elementId}	Pass
IT-FB-03	Handling fehlender Felder	Rendering von N/A bei fehlenden Nameplate-Feldern (z.B. ManufacturerProductDesignation)	Pass
IT-FB-04	Error-Response Handling	Prüfung auf ClientErrorResourceNotFound und error message Struktur	Pass
IT-FB-05	Responsives Layout	Prüfung unterschiedlicher Layouts für Desktop/Mobil	Pass
IT-FB-06	HTTP Error Codes	Validierung von 400/404 Responses mit korrektem statusCode	Pass
IT-FB-07	Loading State	Prüfung visuellen Ladens während asynchroner Datenbeschaffung	Pass

4.4 BaSyx-Integration (DIN EN 18222 konform)

Test-ID	Beschreibung	Ziel	Status
IT-BX-01	DPP zu AAS Transformation	Transformation von DPP <code>info</code> + <code>submodels</code> zu AAS-Struktur mit Asset Information	Pass
IT-BX-02	Registry Entry Management	Vorbereitung und Registrierung von DPPs mit <code>registryIdentifier</code>	Pass
IT-BX-03	Discovery Service Integration	Abruf von DPPs über <code>productId</code> über Discovery Service	Pass
IT-BX-04	Submodel Reference Handling	Prüfung von Semantic IDs und Submodel-Referenzen in AAS	Pass
IT-BX-05	Asset Information Extraction	Extraktion und Verwendung von Asset Information für AAS Creation	Pass

4.5 Auswertungslogik

- Pass: Testfall erfolgreich ausgeführt und fachlich korrekt
- Not Executed: Testfall dokumentiert, aber im aktuellen Stand noch nicht gegen das reale Backend ausgeführt
- Die Tabellen in Abschnitt 3 bilden den Soll-Zustand der Systemtests ab; Abschnitt 4 dokumentiert die aktuell verfügbaren Ausführungen und vorbereiteten Integrationsprüfungen

5. Artefakte und Ablage

Test-Dateien

Die vorbereiteten Integrationstests liegen im Frontend-Projekt unter:

- [SOURCE/frontend/aas-web-ui/tests/integration/DppApiIntegration.test.ts](#) – 14 Tests für DIN EN 18222 API-Endpoints
- [SOURCE/frontend/aas-web-ui/tests/integration/FrontendBackendIntegration.test.ts](#) – 7 Tests für Viewer und API Interaktion
- [SOURCE/frontend/aas-web-ui/tests/integration/BaSyxIntegration.test.ts](#) – 5 Tests für BaSyx AAS Orchestrierung
- [SOURCE/frontend/aas-web-ui/tests/integration/integrationHarness.ts](#) – Geteilte Mock-Backend Implementation und Test Utilities

Setup und Dokumentation

- [SOURCE/frontend/aas-web-ui/REAL_BACKEND_SETUP.md](#) – Anleitung zur Ausführung gegen echtes Backend
- [SOURCE/frontend/aas-web-ui/scripts/setupTestData.mjs](#) – Automatisiertes Skript zur Vorbereitung von Test-Daten im echten Backend

Standardreferenzen

- [PROJECT/SAS/mapping/DPP-Data-Object-Structure.md](#) – Vollständige DIN EN 18222 Datenstrukturspezifikation

Konfiguration und Automation

Zur reproduzierbaren Ausführung im Repository wurden außerdem hinterlegt:

- [SOURCE/frontend/aas-web-ui/package.json](#) mit dem Skript `test:integration`
- [SOURCE/frontend/aas-web-ui/package.json](#) mit dem Skript `test:integration:real` für Real-Backend-Tests (ohne Mocking, mit `RUN_REAL_BACKEND_TESTS=true` und konfigurierbarer `DPP_BACKEND_BASE_URL`)
- [SOURCE/frontend/aas-web-ui/package.json](#) mit `engines.node` zur Vorgabe der unterstützten Node.js-Versionen für WSL und CI

- [.github/workflows/systemtest-issue-tracker.yml](#) zur automatisierten Ausführung der STR-nahen Tests in GitHub Actions

5.1 Real Backend Testing Vorbereitung

Die Tests können nach Backend-Fertigstellung **ohne Code-Änderungen** gegen das echte Backend ausgeführt werden.

Schritt 1: Backend starten Stelle sicher, dass der DPP-Backend auf <http://localhost:8080> läuft (oder konfiguriere `DPP_BACKEND_BASE_URL` für abweichende URLs).

Schritt 2: Test-Daten vorbereiten Führe das Setup-Skript aus:

```
cd SOURCE/frontend/aas-web-ui
node scripts/setupTestData.mjs
# Oder mit benutzerdefinierten Backend-URL:
node scripts/setupTestData.mjs http://backend.example.com:8080
```

Das Skript erstellt automatisch:

- Test-DPP mit ID `urn:uuid:test-dpp-1`
- Product-ID Mapping `urn:uuid:prod-123`
- Registry-Eintrag `aas-registry://dpps/test-dpp-1`
- Submodel-Daten (Nameplate, Technical Data, Carbon Footprint)

Schritt 3: Real-Backend-Tests ausführen

```
yarn test:integration:real
# Oder:
RUN_REAL_BACKEND_TESTS=true DPP_BACKEND_BASE_URL=http://localhost:8080 yarn test:integration
```

Erwartetes Ergebnis: **26/26 Tests bestanden** (API: 14, Frontend: 7, BaSyx: 5)

Weitere Details: Siehe [REAL_BACKEND_SETUP.md](#)

6. Zusammenfassung

Die Systemtests sind inhaltlich aus dem SRS abgeleitet und wurden auf die [DIN EN 18222 DPP-Data-Object-Structure Spezifikation](#) abgebildet. Vorbereitete, automatisierte Integrationstests sind erstellt und arbeiten aktuell mit Mocking. Damit ist eine belastbare, standards-konforme Testbasis vorhanden, obwohl das Backend noch nicht vollständig fertiggestellt ist.

Aktueller Ausführungsstand der vorbereiteten Integrationstests (Vitest): **3 Testdateien, 26/26 Tests bestanden** (API: 14, Frontend-Backend: 7, BaSyx: 5).

Umschaltungsmechanismus für echtes Backend:

- **Standard:** Mock-basierte Tests (`yarn test:integration`) – läuft ohne Backend-Installation
- **Production-Ready:** Real-Backend-Tests (`yarn test:integration:real`) – läuft gegen echten Backend nach Vorbereitung
- **Setup-Automatisierung:** `node scripts/setupTestData.mjs` erstellt Test-Daten automatisch

Mappings und Konformität:

- API-Tests folgen exakt den DIN EN 18222 Endpoints: CreateDPP, ReadDPPById, ReadDPPByProductId, ReadDPPVersionByProductIdAndDate, ReadDPPIdsByProductIds, UpdateDPPById, DeleteDPPById,

PostNewDPPTToRegistry, ReadDataElementCollection, ReadDataElement, UpdateDataElementCollection, UpdateDataElement, sowie Error Responses.

- Typsignaturen korrespondieren mit standardisierten Status Codes, Payload-Strukturen, und Error Message Objekten.
- BaSyx-Tests prüfen Transformation und Orchestrierung gemäß AAS-Datenmodell.

Nächste sinnvolle Schritte:

1. **Backend-Endpunkte** auf die DIN EN 18222 konformen Test-Definitions abgleichen.
2. **Mock-Responses** schrittweise durch echte Backend-Antworten ersetzen (automatisch über Umgebungsvariable).
3. **BaSyx Repository, Registry und Discovery Integration** validieren.
4. **Testergebnisse** nach Ausführung in diesem STR dokumentieren.

Gesamtstatus: Die vorbereiteten System- und Integrationstests sind fachlich aus dem SRS abgeleitet, auf das STP gemappt und in Mock-Mode erfolgreich ausgeführt worden (26/26 vorbereitete Integrationstests). Die realen Backend- und BaSyx-Tests sind vorbereitet, aber in der aktuellen Projektlage noch nicht vollständig gegen das Endsystem abgenommen.